Access Granted MVP Plan

AI Community Sprint • An MVP (Minimum Viable Product) is the smallest version of your app that solves the problem for one real user. Not every feature. Just the one that matters, working.

App name	Access Granted NYC Transit Accessibility Companion	**Date**	November 10, 2026
Team members	Melissa Noel (solo project)	**Teaching Fellow**	Desmend

1. MVP Statement

Pull this from your Scoping Worksheet. One sentence a stranger could understand.

My app helps NYC subway riders with mobility needs understand a station's recorded step-free access before traveling by using AI to explain verified MTA station and entrance information in plain language.

The problem it solves	A rider must connect station accessibility, entrance and platform restrictions, and elevator information before traveling. Unclear or outdated information can lead to stairs, detours, or an abandoned trip. This MVP explains verified station facts and clearly identifies information it cannot confirm.
Why AI? *What does AI do here that normal code can't?*	The station card presents verified facts using ordinary code. AI adds natural-language questions and short explanations grounded in those same records, such as which documented entrance serves a particular direction. It states uncertainty when the evidence is insufficient.

2. Meet Your User

Build for one person. Name them (real or invented) and keep them in mind all sprint.

Who are they? *Name, role, age range, situation.*	**Denise** (invented), 50s, lives in the Bronx and uses a manual wheelchair. She takes the subway to work and appointments, often to stations she doesn't know well.
What are they trying to do? *Their goal in one sentence.*	Know before she leaves home whether she can get in and out of a station step-free, and which entrance to use.
What frustrates them today? *What's hard, slow, or confusing without our app?*	She has to piece together trip directions, MTA station pages and elevator outage notices, then work out what it all means for her. If she guesses wrong she arrives at stairs or a broken elevator and has to backtrack.
What device and setting? *Phone on the bus? Laptop at the library?*	Her phone: at home before a trip, and on the street or platform when plans change. Large text, big buttons and screen-reader-friendly labels matter.

This document contains confidential and proprietary information. It is intended solely for the use of the recipient(s) named and may not be disclosed or distributed without prior written consent.

3. The Core Journey

Walk through the one main path from open to done. 3–5 steps. If a step isn't needed, cut it.

Step	User does…	App shows or does…	AI does…
1	Opens the app and types a station name (e.g. "Borough Hall").	Shows matching pilot stations with borough and subway service labels. Similar station names remain distinct.	Nothing yet.
2	Taps the station.	Shows recorded ADA classification, verified entrance and platform details, source links, verification date, and current elevator status marked as not checked.	Nothing yet (the card is plain data, so it works even if the AI is down).
3	Asks a question: "Can I exit here without stairs?"	Retrieves the selected record through the server and displays a short answer plus one next step.	Uses only the selected verified record. Explains direction restrictions and missing information; never infers live elevator status from a snapshot.
4	Sees a (sample) elevator outage on the card and asks "What should I do?"	Displays a separate sample-outage mode, visibly labeled as demonstration data.	Labels the scenario as a sample. Explains its effect and suggests only verified alternatives included in the data, or checking with MTA.

"Done" looks like: the rider can find a pilot station, understand its recorded access and entrance restrictions, and distinguish verified reference facts from unknown current conditions and sample outages.

4. Features: Pick Your Battles

Brainstorm freely, then sort. Rule of thumb: no more than 3 must-haves. If it's not on the core journey, it isn't a must-have.

MUST HAVE (MVP)	NICE TO HAVE (if time)	NOT NOW (parking lot)
1. Search across 12 proposed pilot station locations or complexes in four boroughs. 2. Accessible station cards with verified facts, dates, sources, and an explicitly labeled sample outage. 3. Gemini Q&A grounded in the selected record with uncertainty and failure handling.	Daily reference-data refresh with validation and retention of the last successful snapshot. Live MTA elevator-status integration. Favorites saved on the device. More stations and voice input.	Full trip routing; bus, commuter rail, and Staten Island Railway coverage; accounts; location tracking; indoor navigation; station photos.

This document contains confidential and proprietary information. It is intended solely for the use of the recipient(s) named and may not be disclosed or distributed without prior written consent.

Pilot coverage is a proposed working list to verify against official MTA records before publication. Manhattan: 34 St–Hudson Yards, 145 St serving the A, Times Sq–42 St. Bronx: West Farms Sq–E Tremont Av, 149 St–Grand Concourse, 161 St–Yankee Stadium. Brooklyn: Atlantic Av–Barclays Center, Jay St–MetroTech, Borough Hall. Queens: Jackson Hts–Roosevelt Av/74 St–Broadway, Court Sq, Queensboro Plaza. One complex may need multiple platform records. The deployed app must show actual verified coverage. A daily refresh is planned; until implemented and tested, the snapshot date is shown honestly.

5. The AI Piece (GAME Plan)

Use the GAME framework from class to plan your AI feature before coding it.

Goal *What is the AI trying to accomplish for the user? One sentence.*	Answer a rider's accessibility question about one station in plain language, using only that station's data, and give a useful next step.
Actions *What can it do?*	Use planned read-only tools search_stations(query) and get_station_accessibility(stationId) to retrieve verified records. Explain those facts and identify uncertainty. Suggest alternatives only when the returned evidence supports them. Tool handlers remain to be implemented.
Memory *What must it remember?*	Keep the selected station and a few follow-up turns in page memory. Clear station-specific history when the station changes and when the page reloads. Avoid persistent storage and logging of raw questions or personal details.
Environment *Where does it run and what does it see?*	A mobile-friendly browser page calls a Node/Express server. The server retrieves the verified station record and calls Gemini using a server-side key. GitHub stores source code; the selected hosting service runs the application.

Input → Output *What goes in and what comes out?*	**Input: a question, selected station ID, and server-retrieved station evidence with source dates and data mode. Output: JSON validated with Zod, { answer, canConfirm, nextStep }. canConfirm means the requested claim is directly supported, not a numerical confidence score or travel guarantee.**
Draft prompt *Write your first instruction to the AI.*	The updated system.md supplies the full prompt. The assistant answers in 2–4 short sentences using verified station evidence, distinguishes platforms and directions, labels demonstration data, and says current elevator status has not been checked unless an implemented live source provides a timestamped result.
When the AI is wrong or unsure *Fallback message? Show sources? Ask a human?*	Keep the factual card visible. Explain missing or ambiguous evidence and provide an official MTA link. If Gemini fails, display a friendly retry message. Reference-data refresh failures retain the last successful snapshot and disclose its date once the refresh feature is implemented.
Guardrails *Topics to avoid, info never to store, length limits.*	Never invent entrances, elevators, alternatives, or current status. Treat user and retrieved text as untrusted instructions. Do not request personal or medical details. Keep secrets on the server, validate requests and outputs, limit question length and API usage, and avoid logging raw conversations.

This document contains confidential and proprietary information. It is intended solely for the use of the recipient(s) named and may not be disclosed or distributed without prior written consent.

6. Tech Stack & Setup

Choose what your team already knows. Boring and working beats fancy and broken.

Layer	My choice	Who owns it?
Frontend	Plain HTML, CSS and JavaScript: one accessible page (search, station card, ask box)	Melissa
Backend / API	Node.js and Express, with read-only station tools and a Gemini question endpoint. Serve frontend assets using the hosting platform's supported layout.	Melissa
Database / data	A dated JSON snapshot of verified MTA station and entrance records plus a separate sample outage. No user database in the initial MVP. A daily refresh later requires durable storage or an automated data update and redeploy.	Melissa
AI model / SDK	Google Gemini, with GEMINI_API_KEY stored in a local ignored .env for development and hosting environment variables for deployment. Zod validates requests and AI output. Configuration and a working NYC-project call remain to be verified.	Melissa
Hosting	Preferred: Vercel connected to the NYC GitHub repository. Replit is an alternative if available. Hosting account choice is pending Melissa's check. A working public link is required by Nov 12; rehearse and keep a backup recording.	Melissa

Setup checklist:

☐ Repo created, everyone has access, and everyone can run it locally

☐ API key stored in a .env file, and .env is in .gitignore

☐ "Hello AI" test works (one call in, one answer out)

7. One-Week Sprint Plan

One concrete task per row, with an owner. Adjust as you go, but protect the demo.

Week 2: Make it work, then make it real

Task	Owner	Done?
Nov 10 — submit revised MVP plan, system.md, and planned tool definitions.	Melissa	☐
Nov 11 — verify pilot data and build search, station card, and Gemini Q&A; deploy a first public version.	Melissa	☐
Nov 12 — complete core journey and failure checks; submit deployed link by 11:59 PM.	Melissa	☐

This document contains confidential and proprietary information. It is intended solely for the use of the recipient(s) named and may not be disclosed or distributed without prior written consent.

Task	Owner	Done?
Nov 13 — test on a real rider's phone and fix the most important problems.	Melissa	☐
Nov 13 — finish README, two introductory slides, and a three-minute script with API usage, risks, and future features.	Melissa	☐
Nov 13 — rehearse twice, record, upload an unlisted YouTube video, and submit before Nov 14 at 12:59 AM.	Melissa	☐

8. Test It & Measure Success

My success measure *Pulled from your scoping goals.*	Initial target: one confirmed tester completes three station-search tasks. Record actual completion time, whether the correct station was chosen, whether she can explain a card's restrictions, and whether she distinguishes a sample outage from real conditions. Check answer accuracy separately against verified source records. Five testers is a future target.
Tester & what confused them?	A friend who regularly rides NYC subways has agreed to test. Her frequent stops are West Farms Sq (2/5), Hudson Yards (7), and 145 St (A). Her mobility needs have not been established. Record feedback after testing and use a pseudonym in submitted notes.
What did they love?	Pending the real-user test. No results claimed yet.
What I'll change	Pending observed feedback. Prioritize errors in station selection, access explanations, and understanding of data dates.

9. Demo-Ready Checklist

Check every box before presenting.

☐ App runs from a live link (or a rehearsed local demo with a backup recording)

☐ Core journey works start to finish, and AI failures show a friendly message, not a crash

☐ No API keys or private data in the repo or on screen, and at least one real user tested it

☐ README done and a 2-minute demo script ready (problem, user, demo, impact)

This document contains confidential and proprietary information. It is intended solely for the use of the recipient(s) named and may not be disclosed or distributed without prior written consent.